Ditsha Live-System Demonstration Guide

This guide demonstrates the implemented Ditsha workflows by interacting with the running citizen application, officer dashboard, API, ML service, and PostgreSQL database.

The strongest evidence is:

1. Interact through the citizen application.
2. Interact through the officer dashboard.
3. Query PostgreSQL immediately afterward.
4. Call the running ML service and show its real output.

1. Start the systems

From PowerShell:

```
cd C:\Users\FCSE24-026\Desktop\ditsha-app

docker compose up -d db ml
```

Start the API in a second terminal:

```
cd C:\Users\FCSE24-026\Desktop\ditsha-app\api
npm run dev
```

Start the officer dashboard in a third terminal:

```
cd C:\Users\FCSE24-026\Desktop\ditsha-app\ditsha-frontend
npm run dev -- --port 5177
```

Start the citizen application in a fourth terminal:

```
cd C:\Users\FCSE24-026\Desktop\ditsha-app\ditsha-flutter
flutter run -d web-server --web-port 8081
```

Open these addresses:

```
Officer dashboard: http://localhost:5177
Citizen app:       http://localhost:8081
API health:        http://localhost:3000/api/health
ML health:         http://localhost:8000/health
```

Show the live health responses:

```
curl.exe http://localhost:3000/api/health
curl.exe http://localhost:8000/health
```

Expected responses include:

```
{"status":"ok"}
```

and:

```
{"status":"ok","service":"ditsha-ml"}
```

2. Demonstrate the citizen workflow

Use the seeded citizen account:

```
Email:    citizen@example.com
Password: CitizenPass123!
```

2.1 Create a draft

In the citizen application:

1. Select a Botswana land board.
2. Select an application type.

3. Enter a description.
4. Save the application as a draft.
5. Show the draft in the citizen application's draft/history area.

2.2 Upload a document

Upload a real PDF or image document and show that it appears against the application.

To demonstrate signature validation, attempt to upload a renamed text file such as `fake.pdf`. The system should reject it because the file contents do not match the claimed PDF type.

2.3 Submit the application

Submit the draft and show the status transition:

```
draft -> submitted
```

2.4 Request information and respond

Leave the citizen application open while the officer requests additional information from the dashboard.

Refresh the citizen application and show:

```
information requested
```

Respond to the request and upload the missing document. Show that the application returns to:

```
under review
```

2.5 Withdraw an active application

Create or use another active application and select withdrawal. Show that the status becomes:

```
withdrawn
```

3. Demonstrate the officer workflow

Open:

```
http://localhost:5177
```

Use the seeded officer account:

```
Email:    officer@example.com
Password: OfficerPass123!
```

Demonstrate:

1. The officer sees the assigned application queue.
2. Search or filter the queue.
3. Open the citizen application.
4. View the application timeline.
5. View uploaded documents.
6. Add an officer note.
7. Request more information.
8. Assign or reassign the application.
9. Override priority if the officer has permission.
10. Approve or reject the application.

The actions should be visible in the dashboard and reflected in the application timeline.

4. Prove the database changed during interaction

After creating the draft, uploading the document, requesting information, responding, and withdrawing, run the following commands.

4.1 Check uploaded documents

```
docker exec -it ditsha-backend-db-1 psql -U postgres -d ditsha -c `
"SELECT document_id,
        application_id,
        original_name,
        document_type,
        version,
        checksum_sha256,
        deleted_at,
        superseded_at
 FROM application_document
 ORDER BY created_at DESC
 LIMIT 10;"
```

The result should contain actual document rows with:

- `version`
- `checksum_sha256`
- `deleted_at`
- `superseded_at`

4.2 Check persisted ML predictions

After requesting ML assistance from the officer dashboard:

```
docker exec -it ditsha-backend-db-1 psql -U postgres -d ditsha -c `
"SELECT prediction_id,
        application_id,
        category,
        score,
        priority_score,
        model_version,
        deterministic,
        created_at
 FROM ml_prediction
 ORDER BY created_at DESC
 LIMIT 10;"
```

This proves that ML assistance results are persisted in the actual database.

4.3 Check the workflow timeline

```
docker exec -it ditsha-backend-db-1 psql -U postgres -d ditsha -c `
"SELECT application_id,
        event_type,
        from_status,
        to_status,
        created_at
 FROM application_event
 ORDER BY created_at DESC
 LIMIT 20;"
```

Expected events can include:

```
draft
submitted
information_requested
under_review
withdrawn
```

4.4 Check information requests

```
docker exec -it ditsha-backend-db-1 psql -U postgres -d ditsha -c `
"SELECT request_id,
        application_id,
        message,
        status,
        requested_at,
```

```
        responded_at
FROM application_information_request
ORDER BY requested_at DESC
LIMIT 10;"
```

This proves that the request-information loop is stored in PostgreSQL rather than simulated only in the frontend.

5. Prove `ml_prediction` replaced `RISK_SCORE`

Run:

```
docker exec -it ditsha-backend-db-1 psql -U postgres -d ditsha -c `
"SELECT to_regclass('public.application_document') AS application_document,
        to_regclass('public.ml_prediction') AS ml_prediction,
        to_regclass('public.risk_score') AS old_risk_score;"
```

Expected result:

```
application_document | application_document
ml_prediction        | ml_prediction
old_risk_score       |
```

The empty `old_risk_score` value proves that the old proposed table does not exist in the current database.

6. Demonstrate the live ML service

Call the running ML endpoint:

```
$mlRequest = @{
    application_id = "live-demo-001"
    application_type = "residential_allocation"
    description = "This is a sufficiently detailed application description for testing."
    document_names = @("identity_document")
    required_documents = @("identity_document", "proof_of_residence")
    duplicate_active_count = 0
    land_board_workload = 50
} | ConvertTo-Json

$mlResponse = Invoke-RestMethod `
    -Uri http://localhost:8000/v1/assist `
    -Method Post `
    -ContentType "application/json" `
    -Body $mlRequest

$mlResponse | ConvertTo-Json
```

Expected output will contain values similar to:

```
{
  "application_id": "live-demo-001",
  "score": 0.5,
  "category": "needs_review",
  "reasons": [
    "Missing document: proof_of_residence"
  ],
  "model_version": "rules-1",
  "deterministic": true,
  "priority_score": 0.3
}
```

The important evidence is:

- The request was sent to the running ML service.
- The response identified the missing document.
- The result is deterministic.
- The ML service is advisory rather than an automatic land decision-maker.

7. Demonstrate the ML leakage correction

Run the old and new experiments separately.

7.1 Old completeness experiment

```
cd C:\Users\FCSE24-026\Desktop\ditsha-app\ml-service
python train_compare.py
```

Show that the old report contains:

```
"target": "label_incomplete"
```

and perfect metrics:

```
"precision": 1.0,
"recall": 1.0,
"f1": 1.0,
"pr_auc": 1.0
```

Explain:

> This result is retained as historical evidence. The label was derived from
>
> the same document-completeness features supplied to the model, so the perfect
>
> score was leakage.

7.2 Current delay experiment

```
python train_delay.py
```

Show:

```
"target": "label_delayed",
"excluded_from_features": [
  "processing_days",
  "label_delayed"
]
```

The current model produces realistic, non-perfect results around:

```
accuracy: approximately 0.74
f1:       approximately 0.80
pr_auc:   approximately 0.89
```

Explain:

> The new target is processing delay. The target and the final
>
> processing-days value are excluded from the input features. The score is no
>
> longer artificially perfect.

8. Recommended live review sequence

Use this exact order during the meeting:

1. Open the citizen app.
2. Create and save a draft.
3. Upload a valid document.
4. Submit the application.
5. Open the officer dashboard.
6. Request information.
7. Refresh the citizen app and respond.
8. Upload the requested document.
9. Return to the officer dashboard.
10. Assign the application.
11. Request ML assistance.

12 Approve or reject the application.

13 Query PostgreSQL for:

- `application_document` - `application_information_request` - `ml_prediction` - `application_event`

1 Call `/v1/assist` directly.

2 Run the old and corrected ML comparison scripts.

This demonstrates the complete system path:

```
Citizen UI
   |
   v
Express API
   |
   v
PostgreSQL
   |
   v
Officer UI
   |
   v
ML service
   |
   v
PostgreSQL audit and timeline records
```

9. Suggested explanation to the supervisor

I am not only showing that the entities and endpoints exist. I will create a
real application, upload a real document, trigger a real request-information
loop, respond as the citizen, invoke the live ML service, and then query
PostgreSQL to show the exact rows and workflow events created by those
interactions.